

MixBytes()

[ SECURITY AUDIT REPORT ]

# DIA Oracle V3

PREPARED FOR DIA

REPORT DATE

September 11, 2026

# Table of Contents

|                                                                                                                   |           |
|-------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                                                                            | <b>4</b>  |
| 1.1 Disclaimer                                                                                                    | 4         |
| 1.2 Executive Summary                                                                                             | 4         |
| 1.3 Project Overview                                                                                              | 7         |
| 1.4 Security Audit Methodology                                                                                    | 10        |
| 1.5 Risk Classification                                                                                           | 12        |
| 1.6 Summary of Findings                                                                                           | 13        |
| <b>2. Findings Report</b>                                                                                         | <b>14</b> |
| 2.1 Critical                                                                                                      | 14        |
| 2.2 High                                                                                                          | 15        |
| H-1 Single oracle with non-zero fairValue flips the sort dimension for all oracles                                | 15        |
| H-2 setDecimals callable by UPDATER_ROLE — precision change silently excludes oracle and corrupts historical data | 16        |
| 2.3 Medium                                                                                                        | 17        |
| M-1 Incorrect median calculation for numerator/denominator ratios                                                 | 17        |
| M-2 Early return in MedianPrice and VWAP methodologies bypasses threshold check                                   | 19        |
| M-3 VolumeWeightedOracleMethodology does not calculate true VWAP                                                  | 20        |
| M-4 ValueStore.setValue accepts numerator=0, denominator=0 — zero denominator can reach consumers                 | 21        |
| M-5 getAggregatedVolume counts zero-volume oracles as valid                                                       | 22        |
| M-6 Equal timestamps accepted — duplicate entries consume ring buffer                                             | 23        |
| M-7 Removing oracles/stores or setting threshold can make threshold unsatisfiable                                 | 24        |
| M-8 ValueStoreRemoved event emits incorrect index after swap                                                      | 25        |
| M-9 Stale rawData persists after setValue overwrites the same key                                                 | 26        |
| 2.4 Low                                                                                                           | 27        |
| L-1 getValue returns fairValue for unrecognized action strings                                                    | 27        |
| L-2 Even-count median timestamp ignores the fresher middle element                                                | 28        |
| L-3 V2-compatible update functions incompatible with VWAP methodology                                             | 29        |
| L-4 Unsafe uint256 to uint128 truncation in volume aggregation and getValue                                       | 30        |
| L-5 Inconsistent rounding in even-count median calculation                                                        | 31        |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| L-6 Code quality issues .....                                         | 32 |
| L-7 Missing __AccessControl_init call in DIAOracleV3.initialize ..... | 33 |
| L-8 Potential overflow in _insertionSort comparison .....             | 34 |
| 3. About MixBytes .....                                               | 35 |

# 1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

## 1.2 Executive Summary

DIA Oracle V3 is a decentralized oracle system consisting of two independent modules: the decentral-feeder (on-chain aggregation of price data from multiple `DIAOracleV3` instances via pluggable methodologies) and the fair-value module (aggregation of fair value, USD value, and numerator/denominator ratios from multiple `ValueStore` contracts via median calculation). Both modules are designed as upgradeable contracts targeting EVM-compatible chains.

The audit identified 2 High, 9 Medium, and 8 Low severity findings. The most impactful issues relate to incorrect median calculation logic in the fair-value module, a sort dimension vulnerability that allows a single misconfigured oracle to corrupt aggregation results, and an overly permissive access control on `setDecimals` that can silently break the entire oracle pipeline.

The codebase was audited in 4 days by 4 auditors using a combination of manual review and automated tooling.

Several additional observations did not warrant standalone findings but are important for the project's long-term health:

**`MAX_HISTORY_SIZE` is immutable and baked into implementation bytecode.** If a new implementation is deployed with a different value (e.g. 200 instead of 100), existing ring buffer arrays in the proxy remain at their original length. The `writeIndex` will exceed the array bounds and `setValue` will revert for all existing keys. This must be accounted for in any upgrade strategy.

**OpenZeppelin dependencies are not pinned.** The project does not build from a clean clone — tests are written against OZ v5.5.0, but `forge install` pulls v5.6.1 which contains breaking changes, causing 3 out of 4 test suites to fail. **Update:** the developers have added a Foundry lock file in both repositories to pin dependency versions.

**A malicious or malfunctioning oracle/ValueStore can DoS the aggregator.** In the fair-value module, arithmetic overflow in the insertion sort comparison ( `keyMain + 1` when `keyMain == type(uint256).max` ) and in the freshness check ( `ts + timeoutSeconds` overflow outside the `try/catch` scope) both cause a revert that cannot be caught. A single compromised ValueStore returning extreme values can permanently block `getMedianValues` for all keys. While this requires a compromised trusted source and is low probability, the lack of defensive handling is worth noting.

**`getValue` returns zeros for nonexistent keys** in both `DIAOracleV3` and `DIAOracleV3MetaFairValueField`. Consumers that do not check the returned timestamp may silently accept a zero price as valid data.

**Oracles with different decimals are filtered out rather than normalized.** `DIAOracleV3Meta` excludes oracles whose `decimals` does not match the meta-contract's setting, rather than rescaling values to a common precision. This is a design choice, but it means adding an oracle with a different decimal configuration silently reduces the effective oracle set without any warning.

**Decimals in `DIAOracleV3MetaFairValueField` are handled by the off-chain service.** All assets and platforms are expected to use the same decimals, so on-chain normalization is not performed.

**Sorting algorithm in `MedianPriceMethodology` is mislabeled.** The comments refer to "bubble sort", but the implementation is actually selection sort (finds the minimum for each position rather than swapping adjacent elements). Either the comments should be corrected or the algorithm replaced with an actual bubble sort or a more gas-efficient alternative like insertion sort.

**Timestamp semantics differ between modules by design.** `DIAOracleV3`, the timestamp is provided by the caller and represents the moment the price was observed at the off-chain source (validated to be within 1 hour of `block.timestamp`). In `ValueStore`, the timestamp was previously set to `block.timestamp` automatically, but is now also provided by the caller and validated via `_validateTimestamp` (within 1 hour of `block.timestamp`, strictly monotonically increasing per key). Since the two modules are fully independent — different metacontracts, different data sources, different consumers — these timestamps never cross module boundaries or get compared to each other. The difference is a natural consequence of the different trust and data models.

**`ValueStore.setValue` now enforces strictly increasing timestamps per key.** The re-audit commit added a caller-provided `timestamp` parameter to `setValue` with a monotonicity check via `_validateTimestamp`. This means that when batch-loading or replaying historical data, entries must be submitted in chronological order – out-of-order submissions will revert. This ordering requirement should be documented for off-chain service operators to avoid unexpected failures during data ingestion.

**Ring buffer data cannot be edited or deleted after submission.** Once a feeder writes a value into the `DIAOracleV3` ring buffer, there is no mechanism to retroactively correct or remove it. If an oracle submits incorrect data, the erroneous entry remains in the history until it is naturally overwritten by newer entries or expires via the timeout. Instead of deleting values, the expected mitigation is to remove or disable the misbehaving feeder/node operator so it stops submitting incorrect data.

**Unbounded operator-submitted values were addressed post-audit in `DIAOracleV3MetaFairValueField`.** During the audit, oracles and `ValueStore` operators were treated as trusted entities expected not to submit erroneous data, so on-chain sanity checks on submitted values were considered redundant. The client subsequently asked to remove the possibility of malformed data affecting aggregation regardless of that trust assumption. The root cause is the same in every case:

`ValueStore` accepts arbitrary `uint256` values, while the metacontract performs unchecked arithmetic on them - `keyMain + 1` in both insertion sort comparisons, `ts + timeoutSeconds` in the freshness check outside the `try/catch` scope, `fairSum += fairValues[i]` when selecting the sort dimension, and `(a + b + 1) / 2` when averaging the two middle elements for an even number of live stores. Any of these reverts outside the `try/catch` scope, so a single extreme value can block `getMedianValues` for all keys. Related to this, the median could be computed from a single surviving non-zero entry once leading zeros were skipped, independently of the configured `threshold`.

**The fixes were implemented entirely in the metacontract and re-reviewed.** They were delivered in [PR #33](#), with the contract changes in commits [c2782b0](#) and [f606814](#) and the accompanying test updates in [91c11cd](#) and [69dd02c](#). Because `ValueStore` is already deployed across multiple nodes, the client chose not to upgrade it and instead applied all validation in `DIAOracleV3MetaFairValueField` - `ValueStore.sol` is unchanged. `_collectValues` now drops any entry whose `fairValue`, `usdValue`, numerator, or denominator exceeds `type(uint128).max`, which matches the `SafeCast.toUint128` already applied downstream, and the freshness check was rewritten as `block.timestamp - ts > timeoutSeconds` with an explicit rejection of future-dated timestamps. Both insertion sort comparisons use `main[prev] <= keyMain`, the sort-dimension selection was reworked to avoid the unbounded sum (and subsequently removed altogether, see below), and even-count averaging goes through a helper that never forms the intermediate sum. The median calculation now reverts with `ThresholdNotMet` when the number of entries actually contributing to the median falls below `threshold`.

The client subsequently removed the implicit sort-dimension fallback entirely in [3f61db0](#) (merged as [73d735b](#)):

`_sortValues` was deleted, `getMedianValues` now always sorts by `fairValue`, and a separate `getMedianValuesByUsd` was added as an explicit usd-sorted entry point. The sort dimension is now chosen by the caller, which is the only party that knows whether a fair value is expected for a given key, rather than inferred from the data. Where a mixed set of values previously switched the whole set to a usd-sorted median silently, `getMedianValues` now reverts with `AllPrincipalEntriesZero`.

We consider the overflow class closed - it is guarded both at the input filter and in the arithmetic itself - and we independently confirmed that the new averaging helper is exactly equivalent to  $(a + b + 1) / 2$  across the full `uint256` range while never forming the intermediate sum. The full test suite passes at the fix commit. Three points remain worth addressing. First, `threshold` is never initialized: the constructor sets only `maxValueStores`, so `threshold` defaults to `0` until an owner calls `setThreshold`, and while it is `0` both the pre-existing `_ensureThresholdMet` check and the newly added median guard are inert. `setThreshold` additionally permits a value of `1`, which allows the same outcome. We recommend initializing `threshold` in the constructor and enforcing a minimum of `2`. Second, the new freshness check rejects any entry with `ts > block.timestamp`, whereas `ValueStore._validateTimestamp` deliberately accepts

timestamps up to `MAX_TIMESTAMP_GAP` (1 hour) into the future. Rows that `ValueStore` considers valid are therefore silently dropped by the metacontract, and under clock skew affecting all feeders simultaneously `getMedianValues` can revert outright. The updated tests encode the new rejection as expected behavior, so the suite does not surface this; the two contracts should be aligned on a single timestamp policy. Third, since out-of-range values are skipped rather than rejected at write time, a call may return a median computed from fewer stores than consumers expect; this should be noted in the documentation and paired with off-chain monitoring of the `ValueUpdated` event to catch bad data at submission time.

**Integrators must handle `getMedianValues` reverting when stores disagree on whether `fairValue` is defined.** Per the client, `fairValue == 0` is a legitimate state rather than bad data - the contract also serves assets for which a fair value is not well-defined (for example, assets backed by several reserve currencies), where only the denominator is non-zero. Stores can therefore legitimately disagree on a single key. Following the removal of the implicit fallback, the sort dimension is no longer inferred from the data: `getMedianValues` always sorts by `fairValue` and reverts `AllPrincipalEntriesZero` when every entry is zero, while `getMedianValuesByUsd` must be called explicitly for assets where only a usd-denominated median is meaningful.

In our opinion reverting is preferable to returning a median derived from one reported value, and that `threshold >= 2` is the right configuration - the residual concern is discoverability rather than the revert itself. Since `getMedianValues` and `getValue` are `view` functions that integrators will treat as always-available, the integration documentation should state that a revert is an expected outcome rather than an outage, that consumers must handle it instead of assuming a value is always returned, which keys are expected to carry a fair value at all. We further recommend off-chain alerting on mixed `fairValue` populations for a single key so the condition is noticed before consumers encounter it, and a distinct error selector to separate this case from ordinary insufficient-data reverts on-chain.

**`getValue` always aggregates by `fairValue`, and is unusable for multi-reserve assets.** `getValue` routes all four actions through `getMedianValues`, so for example `getValue("usdValue:BTC")` returns the `usdValue` from the `fairValue`-median row rather than a usd-sorted median. Because `getMedianValues` reverts with `AllPrincipalEntriesZero` when every entry has `fairValue == 0`, and per the README `fairValue` is non-zero only for single-reserve assets, `getValue` reverts for all four actions - `fairValue`, `usdValue`, `numerator` and `denominator` - on multi-reserve assets.

`getMedianValuesByUsd` is therefore not reachable through `getValue` at all for that asset class. The client confirmed this is intended: an off-chain attester service selects which function to call based on `assetType`, and no multi-reserve assets are currently deployed, so the restriction is not yet exercised. We note that returning a coherent row rather than five independently computed medians is consistent with the client's position on `numerator / denominator` medians elsewhere in this report. Since `getValue` is the ABI-facing entry point and the constraint is recorded only in NatSpec, the integration documentation should state which actions are valid for which keys, and that consumers needing a usd-sorted median must call `getMedianValuesByUsd` directly. This becomes load-bearing the moment the first multi-reserve asset is onboarded: any integrator or future component that reaches for `getValue` by default will revert.

The off-chain feeder infrastructure, deployment scripts, and upgrade migration procedures were out of scope of this audit.

Overall, the codebase is well-structured and modular, but would benefit from pinning dependency versions, adding input validation on admin setters (bounds checks on `threshold`, `windowSize`, `timeoutSeconds`), improving test coverage for edge cases (even-length arrays, zero-volume entries, mixed-decimal oracles), and fixing the issues presented in the report below.

## 1.3 Project Overview

### Summary

|          |                         |
|----------|-------------------------|
| CLIENT   | DIA                     |
| CATEGORY | Oracle                  |
| PROJECT  | DIA Oracle V3           |
| TYPE     | Solidity                |
| PLATFORM | EVM                     |
| TIMELINE | 04.03.2026 - 08.09.2026 |

### Scope of Audit

decentral-feeder/contracts/DIAOracleV3.sol

decentral-feeder/contracts/DIAOracleV3Meta.sol

decentral-feeder/contracts/methodologies/MedianPriceMethodology.sol

decentral-feeder/contracts/methodologies/VolumeWeightedAveragePriceMethodology.sol

fair-value/contracts/metacontract/DIAOracleV3MetaFairValueField.sol

fair-value/contracts/valuestore/ValueStore.sol

## Versions Log

| DATE       | COMMIT HASH                              | NOTE                                   |
|------------|------------------------------------------|----------------------------------------|
| 04.03.2026 | 15e9049f57602bcb673fbcc8006ad6076470f8f4 | Initial Commit (decentral-feeder)      |
| 04.03.2026 | 00bf59d5bd7153a549cedb09712eacd1caf74a2a | Initial Commit (fair-value)            |
| 18.03.2026 | ff2ea102744ba3e027a23234397bbd92aae747d2 | Commit for Re-audit (decentral-feeder) |
| 18.03.2026 | 9cab374be377e55cad72c3d6c55a1a307735bea9 | Commit for Re-audit (fair-value)       |
| 20.03.2026 | 38753f54d6e770e92d2b1d440b1ff9a2f5b96240 | Commit with Updates (decentral-feeder) |
| 20.03.2026 | 0789e7c67a7684b7747bdbf452326a2e0a79ba6f | Commit with Updates (fair-value)       |
| 26.08.2026 | 69dd02ca1ac9cc43c9d369a5940bada0ab73054b | Commit with Updates (fair-value)       |
| 01.09.2026 | 73d735b7ba6898b2068f56d32ed13b544d1f8847 | Commit with Updates (fair-value)       |

# Mainnet Deployments

The deployed contracts – `DIAOracleV3`, `DIAOracleV3Meta`, `MedianPriceMethodology`, `DIAOracleV3MetaFairValueField`, and `ValueStore` – were verified against the audited sources: bytecode and configuration parameters match.

Both ERC1967Proxy deployments – `0xf9AD4E64206F943708dFa6d5eFf0729e4095eD56` (proxying `DIAOracleV3`) and `0x3C3715bc53dD3Ba2E60DE7611F7F6C449641e5F9` (proxying `ValueStore`) – use identical unmodified OpenZeppelin v5.5.0 proxy sources and are not vulnerable to a Clandestine Proxy-in-the-Middle of Proxy attack – the constructor atomically sets the implementation and executes `initialize()` via `delegatecall`, confirmed by Initialized and Upgraded events emitted in each deployment transaction, leaving no window for front-running.

Currently, `timeoutSeconds` and `windowSize` in `DIAOracleV3Meta` are both set to 0, and `timeoutSeconds` in `DIAOracleV3MetaFairValueField` is also set to 0, meaning the meta-oracle will reject all oracle values as expired and return no valid aggregated price until these parameters are configured.

The OpenZeppelin dependencies are not pinned to a stable release tag. We recommend pinning dependencies to a stable version to ensure reproducible builds and consistency between audited and deployed code.

Following the audit, `DIAOracleV3MetaFairValueField` was redeployed at `0x369cb935eD8B445980FCb068860452D38f9A7781` to carry the post-audit fixes from commit `73d735b7ba6898b2068f56d32ed13b544d1f8847`, replacing the earlier deployment at `0x5aCEc1713BD9E00498215950667EEB902410c0CC`. The new contract is a direct non-proxy deployment and is therefore immutable. We verified it against the audited sources: the runtime bytecode matches a build of commit `73d735b7ba6898b2068f56d32ed13b544d1f8847` byte for byte, and of the seven verified source files published for the deployment, six are identical to the repository. The seventh, `DIAOracleV3MetaFairValueField.sol` itself, differs only in NatSpec comments, with no executable change. All post-audit fixes are present in the deployed code, `getMedianValuesByUsd` is deployed and callable, and the live configuration is `threshold` 2, `timeoutSeconds` 86400 and 11 registered ValueStores. Aggregation was additionally checked against live data: the median returned on chain matches an independently computed median of the values reported by the registered stores at the same block.

| FILE                                       | ADDRESS                                                 | BLOCKCHAIN   |
|--------------------------------------------|---------------------------------------------------------|--------------|
| <code>MedianPriceMethodology</code>        | <code>0x551BC8b541304F3C7791d333F70F05CA15232b97</code> | DIA LaserNet |
| <code>DIAOracleV3Meta</code>               | <code>0xBd82Fc0067CA5977b86c6EDB579f90DFcFb62D7d</code> | DIA LaserNet |
| <code>ERC1967Proxy</code>                  | <code>0xf9AD4E64206F943708dFa6d5eFf0729e4095eD56</code> | DIA LaserNet |
| <code>DIAOracleV3</code>                   | <code>0x042e7290E05B5c7E0eC3Eed4247E516f7551Eaf8</code> | DIA LaserNet |
| <code>DIAOracleV3MetaFairValueField</code> | <code>0x369cb935eD8B445980FCb068860452D38f9A7781</code> | DIA LaserNet |
| <code>ERC1967Proxy</code>                  | <code>0x3C3715bc53dD3Ba2E60DE7611F7F6C449641e5F9</code> | DIA LaserNet |
| <code>ValueStore</code>                    | <code>0x0237Af88794780FffE775697f3f889EceB0054D8</code> | DIA LaserNet |

## 1.4 Security Audit Methodology

### Stage 1 Interim Audit

#### 01 PROJECT ARCHITECTURE REVIEW

**OBJECTIVE:** UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

#### 02 ADVERSARIAL CODE REVIEW

**OBJECTIVE:** IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

#### 03 SYSTEMATIC VULNERABILITY ANALYSIS

**OBJECTIVE:** APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

#### 04 CONSOLIDATION OF AUDITORS' REPORTS

**OBJECTIVE:** MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

#### Stage 2

## Re-audit & Mainnet Verification

#### RE-AUDIT

**OBJECTIVE:** VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

#### MAINNET DEPLOYMENT VERIFICATION

**OBJECTIVE:** FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

# 1.5 Risk Classification

## Severity Level Matrix

| SEVERITY           | IMPACT: HIGH | IMPACT: MEDIUM | IMPACT: LOW |
|--------------------|--------------|----------------|-------------|
| LIKELIHOOD: HIGH   | CRITICAL     | HIGH           | MEDIUM      |
| LIKELIHOOD: MEDIUM | HIGH         | MEDIUM         | LOW         |
| LIKELIHOOD: LOW    | MEDIUM       | LOW            | LOW         |

### IMPACT

**HIGH** - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

**MEDIUM** - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

**LOW** - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

### LIKELIHOOD

**HIGH** - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

**MEDIUM** - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

**LOW** - A highly unlikely event that can only be triggered by the contract owner.

### ACTION REQUIRED

**CRITICAL** - Must be fixed as soon as possible.

**HIGH** - Strongly advised to be fixed in order to minimize potential risks.

**MEDIUM** - Recommended to be fixed to improve security and stability.

**LOW** - Recommended to be fixed to improve overall robustness and efficiency.

### FINDING STATUS

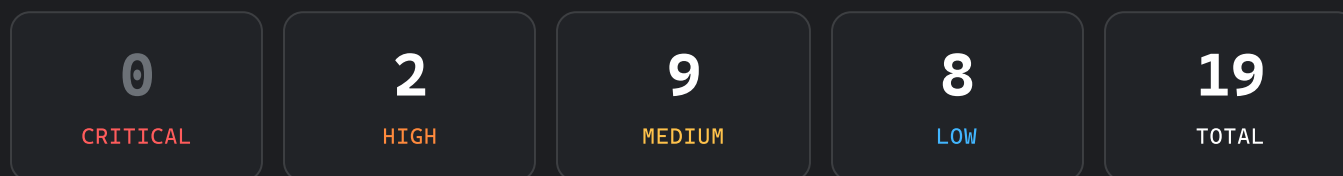
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

## 1.6 Summary of Findings

### Findings Count



### Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

| ID  | FINDING                                                                                                       | SEVERITY              | STATUS                      |
|-----|---------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------------|
| H-1 | Single oracle with non-zero fairValue flips the sort dimension for all oracles                                | <span>● HIGH</span>   | <span>● FIXED</span>        |
| H-2 | setDecimals callable by UPDATER_ROLE — precision change silently excludes oracle and corrupts historical data | <span>● HIGH</span>   | <span>● FIXED</span>        |
| M-1 | Incorrect median calculation for numerator/denominator ratios                                                 | <span>● MEDIUM</span> | <span>● ACKNOWLEDGED</span> |
| M-2 | Early return in MedianPrice and VWAP methodologies bypasses threshold check                                   | <span>● MEDIUM</span> | <span>● FIXED</span>        |
| M-3 | VolumeWeightedOracleMethodology does not calculate true VWAP                                                  | <span>● MEDIUM</span> | <span>● ACKNOWLEDGED</span> |
| M-4 | ValueStore.setValue accepts numerator=0, denominator=0 — zero denominator can reach consumers                 | <span>● MEDIUM</span> | <span>● ACKNOWLEDGED</span> |
| M-5 | getAggregatedVolume counts zero-volume oracles as valid                                                       | <span>● MEDIUM</span> | <span>● FIXED</span>        |
| M-6 | Equal timestamps accepted — duplicate entries consume ring buffer                                             | <span>● MEDIUM</span> | <span>● FIXED</span>        |
| M-7 | Removing oracles/stores or setting threshold can make threshold unsatisfiable                                 | <span>● MEDIUM</span> | <span>● FIXED</span>        |

| ID  | FINDING                                                                 | SEVERITY | STATUS         |
|-----|-------------------------------------------------------------------------|----------|----------------|
| M-8 | ValueStoreRemoved event emits incorrect index after swap                | ● MEDIUM | ● FIXED        |
| M-9 | Stale rawData persists after setValue overwrites the same key           | ● MEDIUM | ● FIXED        |
| L-1 | getValue returns fairValue for unrecognized action strings              | ● LOW    | ● FIXED        |
| L-2 | Even-count median timestamp ignores the fresher middle element          | ● LOW    | ● FIXED        |
| L-3 | V2-compatible update functions incompatible with VWAP methodology       | ● LOW    | ● ACKNOWLEDGED |
| L-4 | Unsafe uint256 to uint128 truncation in volume aggregation and getValue | ● LOW    | ● FIXED        |
| L-5 | Inconsistent rounding in even-count median calculation                  | ● LOW    | ● FIXED        |
| L-6 | Code quality issues                                                     | ● LOW    | ● FIXED        |
| L-7 | Missing __AccessControl_init call in DIAOracleV3.initialize             | ● LOW    | ● FIXED        |
| L-8 | Potential overflow in _insertionSort comparison                         | ● LOW    | ● FIXED        |

## [ 2 FINDINGS REPORT ]

# 2.1 Critical

NO CRITICAL SEVERITY FINDINGS

## 2.2 High

### H-1 Single oracle with non-zero fairValue flips the sort dimension for all oracles

● HIGH

● FIXED

#### DESCRIPTION

In `DIAOracleV3MetaFairValueField._sortValues`, the sort key is chosen globally based on whether `fairSum` is non-zero. If even one `ValueStore` returns a non-zero `fairValue` while others return zero (e.g. because the asset has no fair value in those stores), the entire array is sorted by `fairValues` — a field where most elements are zero. Zero-valued entries cluster at the bottom of the sorted array, and the median position lands on an arbitrary oracle whose result bears no relation to the actual median of the data. The `usdValue`, `numerator`, and `denominator` fields — which are reordered in parallel — end up in an equally meaningless order. A single misconfigured or partially-reporting `ValueStore` controls the sort dimension for the whole aggregation, producing an incorrect oracle output.

#### RECOMMENDATION

Sort by the ratio or by individual oracle fair values rather than using a global sum-based heuristic. Alternatively, filter out oracles that lack a `fairValue` before sorting, so that zero entries do not pollute the median calculation.

#### CLIENT'S COMMENTARY

Fixed by removing leading zeros in median calculation.

5ea74a49

## H-2 `setDecimals` callable by `UPDATER_ROLE` — precision change silently excludes oracle and corrupts historical data

● HIGH

● FIXED

### DESCRIPTION

In `DIAOracleV3`, the `setDecimals` function is gated by `UPDATER_ROLE`. Since `DIAOracleV3Meta.getValue` filters oracles by matching `decimals`, a decimals change silently excludes the oracle from aggregation. If this drops the count of matching oracles below `threshold`, all `getValue` calls start reverting. On top of that, existing historical values in the ring buffer remain stored with the old precision but are now interpreted under the new one, silently corrupting price data for all consumers.

### RECOMMENDATION

We recommend disallowing changing decimals once any values have been written to the oracle.

### CLIENT'S COMMENTARY

fixed as per recommendation

7e1fc667

## 2.3 Medium

### M-1 Incorrect median calculation for numerator/denominator ratios

● MEDIUM

● ACKNOWLEDGED

#### DESCRIPTION

In `DIAOracleV3MetaFairValueField._calculateMedian`, when there is an even number of `ValueStores`, the median of `numerator` and `denominator` is calculated by averaging them independently:

```
uint256 sumNum = nums[mid1] + nums[mid2];
numerator = (sumNum + 1) / 2;

uint256 sumDen = dens[mid1] + dens[mid2];
denominator = (sumDen + 1) / 2;
```

This produces  $(a+c)/(b+d)$ , which is not equal to the median of two ratios  $(a/b + c/d) / 2 = (ad + bc) / (2bd)$ . When the two middle oracles have different scales for their numerator and denominator, the resulting ratio can be meaningfully incorrect.

At the same time, it should be noted that if `numerator` and `denominator` are calculated either as an arithmetic mean or using the formula above, they lose their physical meaning. They no longer represent the amount of assets anywhere or some value in USD.

#### RECOMMENDATION

Compute the median of ratios correctly, e.g. as  $(ad + bc) / (2bd)$ , or document the current behavior as an intentional simplification and its limitations.

Additionally, consider the following approach as an option: instead of calculating `numerator` and `denominator`, use the values from the store with the highest `timestamp`. This would preserve a certain physical meaning.

#### CLIENT'S COMMENTARY

**Client:** The suggestion does not work because  $d$  (and  $b$ ) can be zero for some use-cases. Potentially losing physical meaning is the tradeoff for covering a wide range of use-cases here.

**MixBytes:** In the case where both numerator and denominator are zero ( $0/0$ ), the median could be taken as  $0/0$  as well. In all other cases, the mathematically correct formula  $(ad + bc) / (2bd)$  should be used, since  $b$  and  $d$  are non-zero there. This would preserve correctness without breaking the zero edge case.

**Client:** As discussed:

This is actually not a bug but a feature. I should have mentioned that this contract will be used for a wide range of use cases, not only plain fair-value where  $\text{fairValue} = \text{numerator} / \text{denominator}$ . Instead, it will also cover cases where  $\text{fairValue}$  is not well-defined (and set to 0), and only the denominator will be non-zero. (For instance for assets backed by several different reserve currencies.)

In case numerator and denominator are non-zero and fairValue is as well, you're right that  $\text{median}(\text{numerator})/\text{median}(\text{denominator}) \neq \text{median}(\text{fairValue})$ . However, in this case the client can just use  $\text{median}(\text{fairValue})$  and ignore the quotient.

This is why we need medians of numerator and denominator as independent quantities.

## M-2 Early return in MedianPrice and VWAP methodologies bypasses threshold check

● MEDIUM

● FIXED

### DESCRIPTION

In `MedianPriceMethodology._aggregateMedianMedian` and `VolumeWeightedAveragePriceMethodology.calculateValue`, if the incoming `oracles` array is empty (for instance, because no oracle matched by `decimals` in `DIAOracleV3Meta`), both methods return `(0, block.timestamp)` before the threshold check is reached. Returning a zero price with a valid current timestamp is incorrect — downstream consumers have no way to distinguish this from a legitimate price update, and the threshold mechanism that is supposed to guard against insufficient data is entirely bypassed.

### RECOMMENDATION

Move the threshold check before the early return, or revert explicitly when the oracles array is empty.

### CLIENT'S COMMENTARY

fixed as per recommendation

6627c4b7

### M-3 VolumeWeightedOracleMethodology does not calculate true VWAP

● MEDIUM

● ACKNOWLEDGED

#### DESCRIPTION

Note: `VolumeWeightedOracleMethodology` was not in the original scope of this audit but was reviewed as part of the methodology contracts analysis.

In `VolumeWeightedOracleMethodology._calculateOracleData`, the per-oracle step computes a simple average price ( `priceSum / validCount` ) and then the cross-oracle step weights each oracle's average by its total volume. True VWAP weights each individual price entry by its own volume: `sum(price_i * volume_i) / sum(volume_i)`. When volumes within a single oracle are unevenly distributed across entries (e.g. one entry with high volume at \$100 and another with low volume at \$200), the simple average (\$150) weighted by total volume gives a different — and less accurate — result than weighting each price individually.

#### RECOMMENDATION

Weight each price entry by its individual volume rather than averaging prices first and weighting by total volume afterwards.

#### CLIENT'S COMMENTARY

Smart Contract is not required and complete

#### M-4 `ValueStore.setValue` accepts `numerator=0`, `denominator=0` — zero denominator can reach consumers

● MEDIUM

● ACKNOWLEDGED

##### DESCRIPTION

In `ValueStore.setValue`, the validation only reverts when `numerator != 0 && denominator == 0`. The case where both are zero passes through. These entries represent "no data" (the off-chain scraper zeroes both fields on `nil`), but they participate in `_collectValues` and the median calculation as valid data points. With an even number of oracles, averaging a `(0,0)` entry with a real one can yield `denominator=0` on output, causing division by zero in downstream consumers that use the ratio.

##### RECOMMENDATION

Either filter out entries with `denominator == 0` in `_collectValues`, or reject `(0,0)` pairs in `ValueStore.setValue`.

##### CLIENT'S COMMENTARY

**Client:** This is on purpose and serves some edge cases needed for initialization of products based on the fair-value feed.

**MixBytes:** If the intent is to allow `denominator=0` for initialization edge cases while continuing to use the  $(a+c)/(b+d)$  averaging formula, then in the case where both numerator and denominator are zero (0/0), the median could be taken as 0/0 as well. In all other cases, the mathematically correct formula  $(ad + bc) / (2bd)$  should be used, since `b` and `d` are guaranteed to be non-zero there. This would prevent a zero denominator from propagating to consumers through the averaging logic while still supporting the initialization use case.

## M-5 `getAggregatedVolume` counts zero-volume oracles as valid

● MEDIUM

● FIXED

### DESCRIPTION

In `DIAOracleV3Meta.getAggregatedVolume`, oracles whose latest value has `volume=0` (e.g. those updated via V2 functions) are still counted in `validOracleCount`. The returned count is meant to indicate how many oracles contributed to the aggregated volume, but it includes oracles that contributed nothing. Consumers relying on this count to assess data quality or quorum are misled into believing more sources provided volume data than actually did.

### RECOMMENDATION

Skip oracles with zero volume when incrementing `validOracleCount`, or document that the count reflects oracles with fresh data regardless of whether they contributed volume.

### CLIENT'S COMMENTARY

fixed as per recommendation

4e353330

## M-6 Equal timestamps accepted — duplicate entries consume ring buffer

● MEDIUM

● FIXED

### DESCRIPTION

In `DIAOracleV3._validateTimestamp`, the monotonicity check uses strict less-than: `if (timestamp < existingTimestamp)`. This means `timestamp == existingTimestamp` is accepted. Each such call adds a new entry to the ring buffer. If an off-chain updater has retry logic and resubmits data with the same timestamp, the buffer fills with duplicates, evicting real price history. The NatSpec states "monotonically increasing" but the implementation enforces "non-decreasing".

### RECOMMENDATION

Use `<=` instead of `<` in the timestamp comparison to enforce strictly increasing timestamps as documented.

### CLIENT'S COMMENTARY

fixed as per recommendation

ffd8915d

## M-7 Removing oracles/stores or setting threshold can make threshold unsatisfiable

● MEDIUM

● FIXED

### DESCRIPTION

In `DIAOracleV3Meta.removeOracle`, `DIAOracleV3MetaFairValueField.removeValueStore`, and both contracts' `setThreshold`, there is no validation that `threshold <= numOracles / numValueStores` after the operation. If the owner removes an oracle when `threshold == numOracles`, or sets `threshold` above the current count, every subsequent `getValue` / `getMedianValues` call reverts with `ThresholdNotMet`. The system becomes fully inoperable until the owner either adds a new source or lowers the threshold.

### RECOMMENDATION

Add a guard in `removeOracle` / `removeValueStore` that reverts if the remaining count would fall below `threshold`. Similarly, cap `setThreshold` at the current oracle/store count.

### CLIENT'S COMMENTARY

**Client:** fixed as per recommendation

83aa16d4

**MixBytes:** The fix was only applied in the `decentral-feeder` (`DIAOracleV3Meta`). The same issue remains in the `fair-value` - `DIAOracleV3MetaFairValueField.removeValueStore` and `setThreshold` still lack the guard, so removing a ValueStore or raising the threshold can still make it unsatisfiable.

**Client:** Added same check in fair value field

b0c984ae

## M-8 ValueStoreRemoved event emits incorrect index after swap

● MEDIUM

● FIXED

### DESCRIPTION

In `DIAOracleV3MetaFairValueField.removeValueStore`, when a store is removed, the last element is swapped into the removed position and the array is shortened. The `ValueStoreRemoved` event is emitted with the index parameter, but at that point the index now refers to the swapped-in store, not the one that was removed. Off-chain indexers and monitoring tools that rely on this event to track which store was removed will associate the wrong store with the removal.

### RECOMMENDATION

Emit the event before performing the swap, or include both the removed store address and the index in the event for unambiguous identification.

### CLIENT'S COMMENTARY

fixed as per recommendation

66c576e1

## M-9 Stale rawData persists after setValue overwrites the same key

● MEDIUM

● FIXED

### DESCRIPTION

In `DIAOracleV3`, calling `setRawValue` stores arbitrary bytes in the `rawData` mapping for a given key. If `setValue` is subsequently called for the same key, it updates the price history but does not clear the associated `rawData`. The old raw data remains accessible via `getRawData`, creating a mismatch between the current price entry and the stale raw payload. Consumers or off-chain services that correlate `getRawData` with the latest value will operate on outdated information.

### RECOMMENDATION

Clear `rawData[key]` inside `setValue` and `setMultipleValues`, or document that raw data is independent of the value history and may become stale.

### CLIENT'S COMMENTARY

Raw Data is cleared as per recommendation for v2 update function

[a50f1bbd](#)

## 2.4 Low

### L-1 `getValue` returns `fairValue` for unrecognized action strings

● LOW

● FIXED

#### DESCRIPTION

In `DIAOracleV3MetaFairValueField.getValue`, if the action portion of the key does not match any of the four known hashes (`fairValue`, `usdValue`, `numerator`, `denominator`), the function silently falls through and returns `fairValue`. A consumer querying a mistyped key like `"farValue:BTC/USD"` receives a valid-looking response with no indication that the requested field is not what they intended.

#### RECOMMENDATION

Revert with a descriptive error when the action hash does not match any known field.

#### CLIENT'S COMMENTARY

Revert with invalid action when action hash does not match  
`f339e59a`

## L-2 Even-count median timestamp ignores the fresher middle element

● LOW

● FIXED

### DESCRIPTION

In `DIAOracleV3MetaFairValueField._calculateMedian`, the comment says "Use max timestamp between selected one" but the code unconditionally takes `timestamps[mid2]` without comparing it to `timestamps[mid1]`. After sorting by `fairValue`, timestamps are reordered as a parallel array — `mid2` corresponds to the higher `fairValue`, not necessarily the higher timestamp. When `mid1` is fresher than `mid2`, the returned timestamp is staler than it should be, which can falsely trigger staleness checks in downstream protocols.

### RECOMMENDATION

Take `max(timestamps[mid1], timestamps[mid2])` as the comment already describes.

### CLIENT'S COMMENTARY

use max timestamp, as per recommendation

[a56acca1](#)

### L-3 V2-compatible update functions incompatible with VWAP methodology

● LOW

● ACKNOWLEDGED

#### DESCRIPTION

In `DIAOracleV3`, `setValue` and `setMultipleValues` store entries with `volume=0` for backward compatibility. `VolumeWeightedAveragePriceMethodology._calculateOracleVWAP` skips entries with zero volume. If an oracle is updated exclusively through V2 functions, all its entries are skipped, the oracle is marked invalid, and it drops out of the aggregation entirely. When enough oracles use V2 functions, the valid count falls below `threshold` and `getValue` reverts. The system provides no error or warning about this incompatibility — the failure surfaces only as a generic `ThresholdNotMet`.

#### RECOMMENDATION

Document the requirement for volume data when using the VWAP methodology. Consider adding a distinct revert (e.g. `NoVolumeData`) when all oracles return `valid=false` due to missing volume, to distinguish this case from genuinely insufficient data.

#### CLIENT'S COMMENTARY

This is expected. VWAP will be used once all V2 contracts are upgraded to V3.

#### L-4 Unsafe uint256 to uint128 truncation in volume aggregation and getValue

● LOW

● FIXED

##### DESCRIPTION

In `DIAOracleV3Meta.getValueWithVolume`, the accumulated `volumeSum` (`uint256`) is cast to `uint128` via `totalVolume = uint128(volumeSum)` without an overflow check. Solidity 0.8.x does not revert on explicit type conversions — if the sum exceeds `type(uint128).max`, the value is silently truncated to the lower 128 bits. The same pattern appears in `getAggregatedVolume`. In `DIAOracleV3MetaFairValueField.getValue`, median results computed as `uint256` are returned as `uint128` with the same unchecked cast.

##### RECOMMENDATION

Use `SafeCast.toUint128()` or add an explicit `require(volumeSum <= type(uint128).max)` before the cast.

##### CLIENT'S COMMENTARY

**Client:** Fixed as per recommendation [2991338a](#)

**MixBytes:** The fix was only applied in the `decentral-feeder` (`DIAOracleV3Meta`). The unsafe `uint256` to `uint128` cast in `DIAOracleV3MetaFairValueField.getValue` in the `fair-value` module remains unfixed.

**Client:** SafeCast added in fair value field  
[26b35c98](#)

## L-5 Inconsistent rounding in even-count median calculation

● LOW

● FIXED

### DESCRIPTION

In `MedianPriceMethodology` and `VolumeWeightedAveragePriceMethodology`, when the number of values is even, the median is computed as  $(a + b) / 2$ , which rounds down. In contrast, `DIAOracleV3MetaFairValueField._calculateMedian` uses  $(sum + 1) / 2$ , which rounds up. The practical impact is minimal (1 wei difference), but the inconsistency across modules means the same input data can produce different prices depending on which aggregation path is used.

### RECOMMENDATION

Use the same rounding approach ( $(sum + 1) / 2$ ) across all median calculations for consistency.

### CLIENT'S COMMENTARY

Use round up in every methodologies

b8095be9

## L-6 Code quality issues

● LOW

● FIXED

### DESCRIPTION

Several code quality issues were identified across the codebase:

**Incorrect storage layout comment in `DIAOracleV3`:** `MAX_HISTORY_SIZE` is listed as occupying slot 0, but it is declared `immutable` and lives in bytecode, not storage. The `decimals` variable (which does occupy storage) is missing from the comment. A developer planning a storage-compatible upgrade could misidentify slot boundaries.

**`getRawDataFromOracle` uses wrong error name:** reverts with `InvalidHistoryIndex` when the actual issue is an invalid oracle index. Confusing for debugging.

**Unused `ThresholdNotMet` error in `DIAOracleV3Meta`:** declared but never used — threshold checks happen inside methodology contracts.

**Unused `MAX_ALLOWED_HISTORY_SIZE` constant in `DIAOracleV3`:** declared but never referenced. Dead code.

**Ring buffer first-write gas cost:** `_addToHistory` allocates 100 `ValueEntry` structs via `push` in a loop on first write (~300 SSTORE to new slots, ~6M gas). Consider lazy initialization or a separate admin function to pre-allocate.

**Double iteration over oracles in `DIAOracleV3Meta.getValue`:** two passes with external `getDecimals()` calls (once to count, once to populate) instead of a single pass with a dynamic array.

**`uint128` loop counter in `setMultipleValues`:** `keys.length` is `uint256` but the loop variable `i` is `uint128`. Silently wraps for arrays longer than `type(uint128).max`. Unreachable in practice due to gas limits, but inconsistent with all other loops in the contract.

### RECOMMENDATION

Address each item individually: fix the storage layout comment, rename the error, remove dead code, and consider gas optimizations for buffer initialization and oracle iteration.

### CLIENT'S COMMENTARY

Fixed storage slot comment

2408d278

Added new error `InvalidOracleIndex`

37b1fe87

Removed unused error

06389114

Removed `MAX_ALLOWED_HISTORY_SIZE`

f3a35c65

use `uint256` as iterator

ed33d5b5

removed double iteration

854f71d2

## L-7 Missing `__AccessControl_init` call in `DIAOracleV3.initialize`

● LOW

● FIXED

### DESCRIPTION

`DIAOracleV3.initialize` does not call `__AccessControl_init()`. While this function is currently a no-op in OpenZeppelin v5.5, omitting it breaks the standard upgradeable initialization convention and creates a risk if a future version adds logic to it. Note: `__UUPSUpgradeable_init()` no longer exists in OpenZeppelin v5.5 (the contract is no longer transpiled), so its absence is correct.

Additionally, both repositories import `UUPSUpgradeable` and `Initializable` from `@openzeppelin/contracts-upgradeable`, which in v5.5 are merely aliases redirecting to `@openzeppelin/contracts`. These contracts were moved to the non-upgradeable package because `UUPSUpgradeable` is stateless (only an immutable) and `Initializable` uses ERC-7201 namespaced storage that does not require the transpilation process. Per the OpenZeppelin v5.5.0 changelog, these aliases will be removed in the next major release. The imports should be updated to reference `@openzeppelin/contracts` directly to avoid breakage on upgrade.

### RECOMMENDATION

Add `__AccessControl_init()` to `DIAOracleV3.initialize`. Update imports of `UUPSUpgradeable` and `Initializable` to use `@openzeppelin/contracts` instead of `@openzeppelin/contracts-upgradeable`, as the aliases are deprecated and will be removed.

### CLIENT'S COMMENTARY

Address accesscontrol init and fixed import [38753f54](#)

<https://github.com/diadata-org/fair-value/commits/oracleV3auditfix/>

## L-8 Potential overflow in `_insertionSort` comparison

● LOW

● FIXED

### DESCRIPTION

In `DIAOracleV3MetaFairValueField._insertionSort`, the comparison `main[prev] < keyMain + 1` can overflow if `keyMain` equals `type(uint256).max`. Although this value is unlikely under normal operation, a single compromised or malfunctioning `ValueStore` returning `type(uint256).max` as a fair value would cause an arithmetic overflow in the sort, reverting `getMedianValues` for all keys and effectively DoS-ing the entire fair-value aggregation.

### RECOMMENDATION

Replace `main[prev] < keyMain + 1` with the equivalent `main[prev] <= keyMain`, which avoids the addition entirely and eliminates the overflow risk.

### CLIENT'S COMMENTARY

Added this change as per recommendation

0558b2e0

# Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

**\$50B+**

AUDITS DELIVERED

**300+**

WEB3 PROTOCOLS

**80+**

## Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

### Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

### AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

### Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

### Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

### CONTACTS

[hello@mixbytes.io](mailto:hello@mixbytes.io)